

7.3 The Magic Square

Definition

In recreational mathematics, a *magic square* is a $(n \cdot n)$ square grid (matrix) filled with distinct positive integers in the range $1, 2, \dots, n$ such that each cell contains a different integer and the sum of the integers in each row, column and diagonal is equal. As the

The sum is called the *magic constant* or *magic sum* of the magic square. As the

square includes all numbers from 1 to $n \cdot n = n^2$ each row, column, and diagonal has to have a sum equal to:

$$(1 + 2 + 3 + \dots + n^2) \cdot \frac{1}{n} = \frac{n^2 \cdot (n^2 + 1)}{2} \cdot \frac{1}{n} = \frac{n \cdot (n^2 + 1)}{2}$$

For example, for a $(3 \cdot 3)$ square, the magic sum is $3 \cdot (9 + 1)/2 = 15$.

Specifications

Write a program that, given a positive integer value n , is able to print-out the magic square of size $(n \cdot n)$.

Example 7.3 Let n be equal to 3. A possible magic square is the following one:

```
2 7 6
9 5 1
4 3 8
```

Solution 1

Let us suppose that the matrix has a size equal to n rows and n columns. First of all, the program generates a permutation of the numbers included in the range $[1, n \cdot n]$. After that, it checks whether this permutation represents a solution to the given problem. As the check is run only when each permutation is generated, and as for a square of size $n \cdot n$ up to $(n \cdot n)!$ permutations have to be generated, the program is unable to find a solution in reasonable times already for very small squares. For example, for a square of size equal to 4 the program has to generate up to $16!$ permutations, i.e., about $2 \cdot 10^{13}$ permutations. If we suppose that one permutation can be generated in $1 \cdot 10^{-9}$ sec the program may run for about 5.5 hours, whereas if each permutation is generated in $1 \cdot 10^{-6}$ sec the program may run up to about 230 days.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  /* function prototypes */
5  int square_r (int **, int, int *, int);
6  int check (int **, int);
7
8  /*
9   * main program
10  */
11 int main (int argc, char *argv[]) {
12     int **matrix, *used, i, j, n;
13
14     n = atoi (argv[1]);
15
16     used = (int *)malloc((n*n+1) * sizeof(int));
17     matrix = (int **)malloc(n * sizeof(int *));
18     if (used==NULL || matrix==NULL) {
19         fprintf (stderr, "Memory allocation error.\n");
20         exit(EXIT_FAILURE);
21     }
22     for (i=0; i<n; i++) {
```

```

23     matrix[i] = (int *)malloc(n*sizeof(int));
24     if (matrix[i]==NULL) {
25         fprintf (stderr, "Memory allocation error.\n");
26         exit(EXIT_FAILURE);
27     }
28 }
29 for (i=0; i<=n*n; i++) {
30     used[i] = 0;
31 }
32
33 if (square_r(matrix, n, used, 0)) {
34     fprintf(stdout, "Magic square of size %d:\n", n);
35     for (i=0; i<n; i++) {
36         for (j=0; j<n; j++) {
37             fprintf(stdout, "%2d ", matrix[i][j]);
38         }
39         fprintf(stdout, "\n");
40     }
41 } else {
42     fprintf(stdout, "Magic Square NOT found!\n");
43 }
44
45 free(used);
46 for (i=0; i<n; i++) {
47     free(matrix[i]);
48 }
49 free(matrix);
50
51 return EXIT_SUCCESS;
52 }
53
54 /*
55  * compute a magic square of given size; return 1 upon success
56  */
57 int square_r (int **matrix, int dim, int *used, int k) {
58     int i, j, val;
59
60     if (k == dim*dim) {
61         return check(matrix, dim);
62     }
63
64     i = k/dim;
65     j = k%dim;
66     for (val=1; val<=dim*dim; val++) {
67         if (used[val] == 0) {
68             /* put a new value and recur */
69             matrix[i][j] = val;
70             used[val] = 1;
71             if (square_r(matrix, dim, used, k+1)){
72                 /* solution found: stop the process */
73                 return 1;
74             }
75             /* backtrack */
76             used[val] = 0;
77         }
78     }
79
80     /* solution not found */
81     return 0;

```

```

82  }
83
84  /*
85   * check whether a given matrix is a magic square; return 1 if yes
86   */
87  int check (int **matrix, int dim) {
88      int i, j, sum, target;
89
90      target = dim*(dim*dim+1)/2;
91
92      /* rows */
93      for (i=0; i<dim; i++) {
94          sum = 0;
95          for (j=0; j<dim; j++) {
96              sum += matrix[i][j];
97          }
98          if (sum != target) {
99              return 0;
100          }
101      }
102
103      /* columns */
104      for (j=0; j<dim; j++) {
105          sum = 0;
106          for (i=0; i<dim; i++) {
107              sum += matrix[i][j];
108          }
109          if (sum != target) {
110              return 0;
111          }
112      }
113
114      /* diagonals */
115      sum = 0;
116      for (i=0; i<dim; i++) {
117          sum += matrix[i][i];
118      }
119      if (sum != target) {
120          return 0;
121      }
122
123      sum = 0;
124      for (i=0; i<dim; i++) {
125          sum += matrix[i][dim-1-i];
126      }
127      if (sum != target) {
128          return 0;
129      }
130
131      return 1;
132  }

```

Solution 2

This second solution check if this condition is verified once each row is generated. Run-times improve a lot and a 4 · 4 magic square can be generated in a fraction of a second.

```
1  #include <stdio.h>
```

```

1  #include <stdlib.h>
2
3  /* function prototypes */
4  int square_r (int **, int, int, int *, int);
5  int check (int **, int, int);
6  int check_runTime (int **, int, int, int, int, int);
7
8
9  /*
10   * main program
11   */
12  int main(int argc, char *argv[]) {
13      int **matrix, *used, i, j, n, target;
14
15      n = atoi (argv[1]);
16      target = n*(n*n+1)/2;
17
18      used = (int *)malloc((n*n+1) * sizeof(int));
19      matrix = (int **)malloc(n * sizeof(int *));
20      if (used==NULL || matrix==NULL) {
21          fprintf(stdout, "Memory allocation error.\n");
22          exit(EXIT_FAILURE);
23      }
24      for (i=0; i<n; i++) {
25          matrix[i] = (int *)malloc(n*sizeof(int));
26          if (matrix[i]==NULL) {
27              fprintf(stdout, "Memory allocation error.\n");
28              exit(EXIT_FAILURE);
29          }
30      }
31      for (i=0; i<=n*n; i++) {
32          used[i] = 0;
33      }
34
35      if (square_r(matrix, target, n, used, 0)) {
36          fprintf(stdout, "Magic square of size %d:\n", n);
37          for (i=0; i<n; i++) {
38              for (j=0; j<n; j++) {
39                  fprintf(stdout, "%2d ", matrix[i][j]);
40              }
41              fprintf(stdout, "\n");
42          }
43      } else {
44          fprintf(stdout, "Magic Square NOT found!\n");
45      }
46
47      free(used);
48      for (i=0; i<n; i++) {
49          free(matrix[i]);
50      }
51      free(matrix);
52
53      return EXIT_SUCCESS;
54  }
55
56  /*
57   * compute a magic square of given size; return 1 upon success
58   */
59  int square_r (int **matrix, int target, int dim, int *used, int k) {
60      int i, j, val;

```

```

61
62     if (k == dim*dim) {
63         return check(matrix, target, dim);
64     }
65
66     i = k/dim;
67     j = k%dim;
68     for (val=1; val<=dim*dim; val++) {
69         if (used[val] == 0) {
70             /* put a new value and recur */
71             matrix[i][j] = val;
72             if (check_runTime (matrix, target, i, j, dim) == 0) {
73                 continue;
74             }
75             used[val] = 1;
76             if (square_r(matrix, target, dim, used, k+1)) {
77                 /* solution found: stop the process */
78                 return 1;
79             }
80             /* backtrack */
81             used[val] = 0;
82         }
83     }
84
85     /* solution not found */
86     return 0;
87 }
88
89 /*
90  * check whether a given matrix is a magic square; return 1 if yes
91  */
92 int check (int **matrix, int target, int dim) {
93     int i, j, sum;
94
95     /* rows */
96     for (i=0; i<dim; i++) {
97         sum = 0;
98         for (j=0; j<dim; j++) {
99             sum += matrix[i][j];
100         }
101         if (sum != target) {
102             return 0;
103         }
104     }
105
106     /* columns */
107     for (j=0; j<dim; j++) {
108         sum = 0;
109         for (i=0; i<dim; i++) {
110             sum += matrix[i][j];
111         }
112         if (sum != target) {
113             return 0;
114         }
115     }
116
117     /* diagonals */
118     sum = 0;
119     for (i=0; i<dim; i++) {

```

```

120     sum += matrix[i][i];
121 }
122 if (sum != target) {
123     return 0;
124 }
125 sum = 0;
126 for (i=0; i<dim; i++) {
127     sum += matrix[i][dim-1-i];
128 }
129 if (sum != target) {
130     return 0;
131 }
132
133 return 1;
134 }
135
136 /*
137  * check whether a given matrix is a magic square; return 1 if yes
138  */
139 int check_runTime (int **matrix, int target, int r, int c, int dim) {
140     int i, j, sum;
141
142     sum = 0;
143     for (j=0; j<=c; j++) {
144         sum += matrix[r][j];
145     }
146     if (c < (dim-1)) {
147         if (sum > target) {
148             return 0;
149         }
150     }
151     else {
152         if (sum != target) {
153             return 0;
154         }
155     }
156
157     sum = 0;
158     for (i=0; i<=r; i++) {
159         sum += matrix[i][c];
160     }
161     if (r < (dim-1)) {
162         if (sum > target) {
163             return 0;
164         }
165     }
166     else {
167         if (sum != target) {
168             return 0;
169         }
170     }
171
172     return 1;

```